演習：Gitの操作(基本編)

慶應義塾大学理工学部物理情報工学科
渡辺

今回の目標

- Gitの初期設定を行う
- コマンドラインでGitの操作を一通り確認する
- VS CodeからもGitの操作ができることを確認する

課題の進め方

「計算物理屋のためのGit/GitHub入門」のサイトにアクセス

https://cpss2026-git.github.io/

計算物理屋のためのGit/GitHub入門

website cpss2026-git.github.io　repo GitHub　org cpss2026-git

概要

計算物理春の学校 2026の共通講義「計算物理屋のためのGit/GitHub入門」のためのサイトです。本リポジトリは、学部3年生用の講義ノートGitHub演習の内容をもとに作成しています。

座学

対応する回にアクセス

4. Gitの基本操作 ← ここにアクセス

- 講義スライド ← 今見ているスライドはこちら
- Gitの初期設定
- ファイルの修正
- 自動ステージング
- VSCodeからの操作
- Gitの操作応用編

課題の進め方

課題: Gitの基本操作

Step 1: 初期設定

ターミナルを開き、以下を実行せよ。

```
git config --global user.name "ユーザー名"
git config --global user.email "メールアドレス"
```

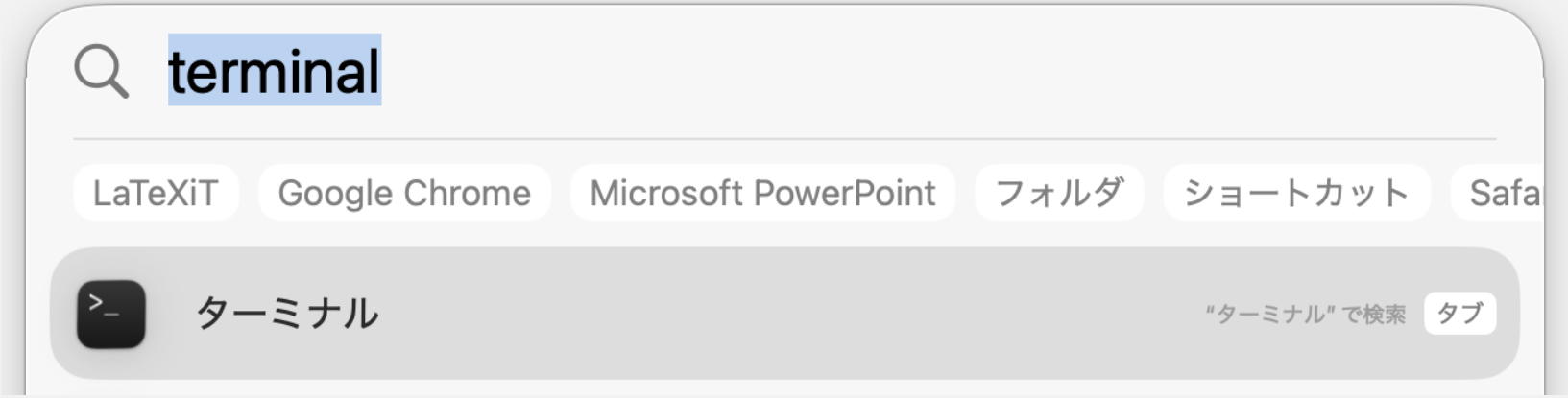
この名前とメールアドレスは後に公開されるため、イニシャルやニックネームなどでも良い。また、ここで「タブ補完」が効くことも確認しておこう。

ここをターミナルに入力し、実行していく

コピペも可能だが、手で入力することを強く推奨
ヒストリ機能(カーソルの上下)やタブ補完機能を活用すること

ターミナルの開き方 (Mac)

Command + スペースキーでSpotlight検索を呼び出す
そこに「terminal」もしくは「ターミナル」と入力

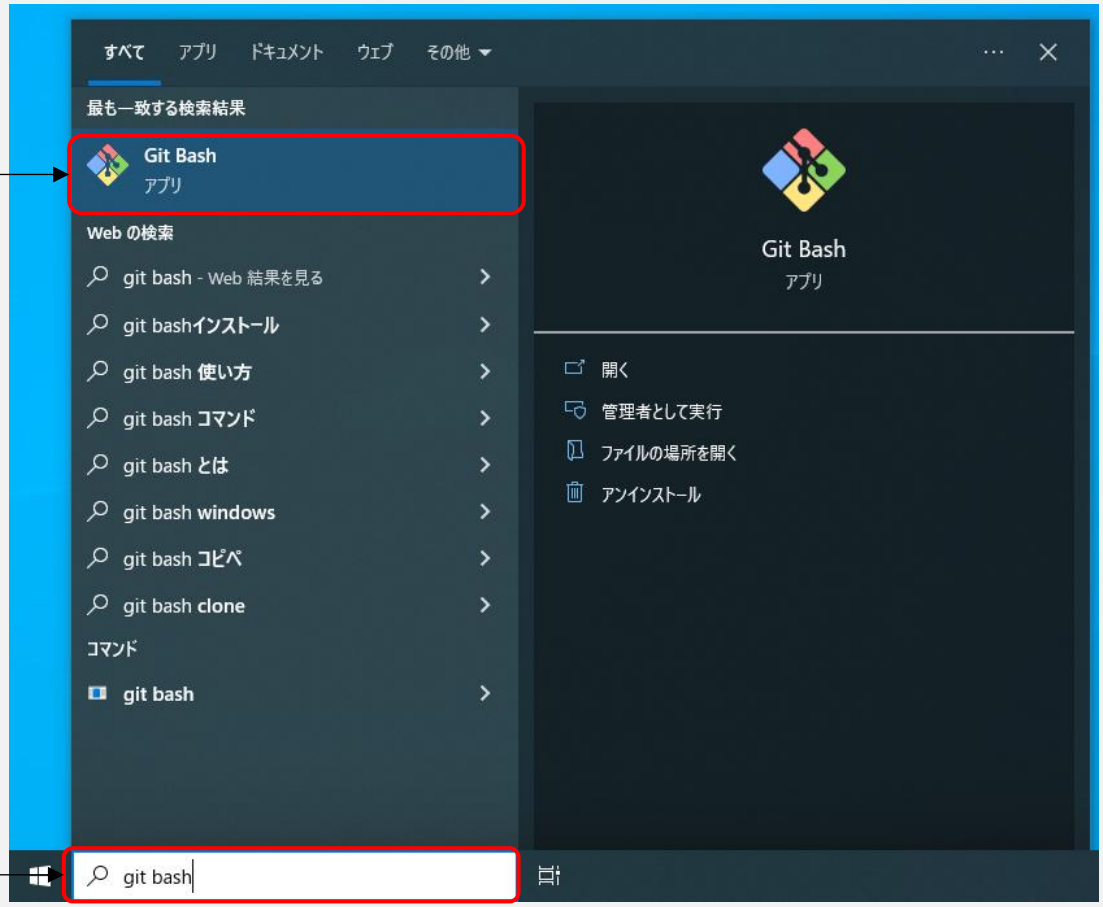


ターミナルの開き方 (Ubuntu)

1. 検索ボックスにubuntuと入力
2. 表示された「Ubuntu 24.04.1 LTS」をクリック

ターミナルの開き方 (Git Bash)

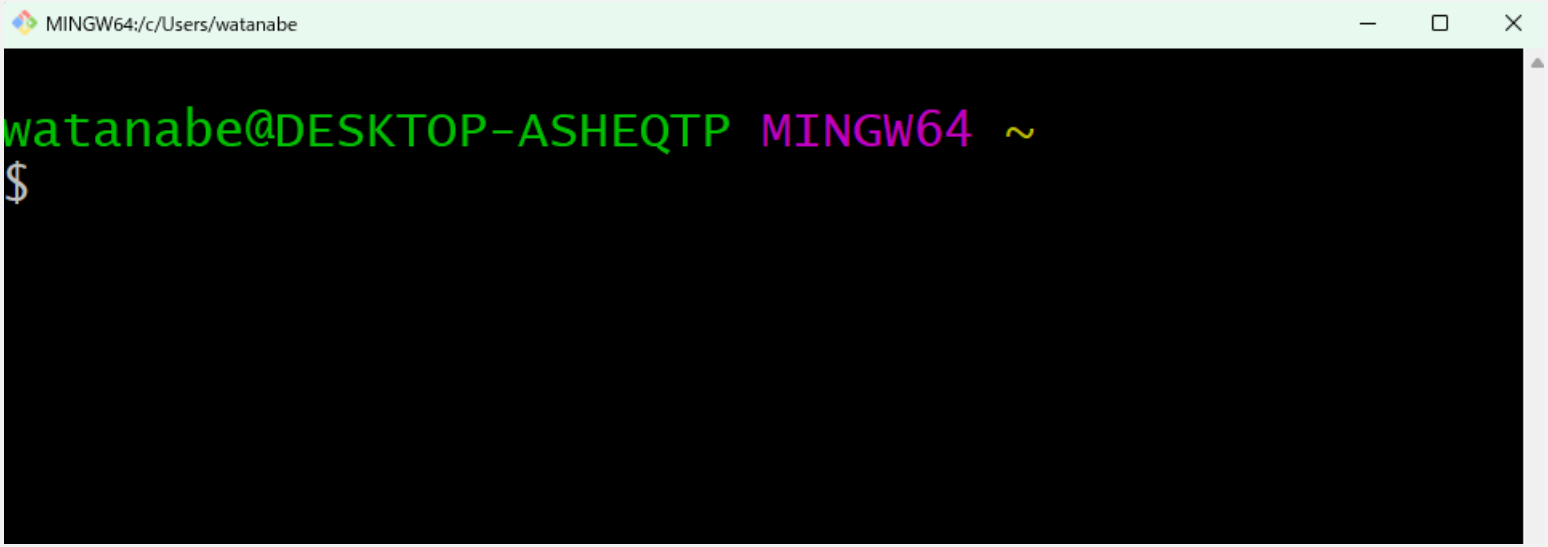
1. 検索ボックスにgit bashと入力
2. 表示された「Git Bash」をクリック



以下、Git Bashを前提に説明を進める

Git Bashとプロンプト

Git Bashを開くと、以下のような画面となる



$マークの右にカーソルが点滅しており、コマンド入力待ちになる
このマークをコマンドプロンプト(もしくは単にプロンプト)と呼ぶ

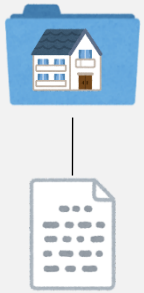
課題 – Step 1

初期設定

Gitに名前やアドレス、その他設定を伝える

```
git config --global user.name "ユーザー名"
git config --global user.email "メールアドレス“
git config --global core.editor vim
git config --global core.autocrlf false
git config --global init.defaultBranch main
git config --global alias.st "status -s"
```

上記設定は、ホームディレクトリ直下の
.gitconfigに書き込まれる



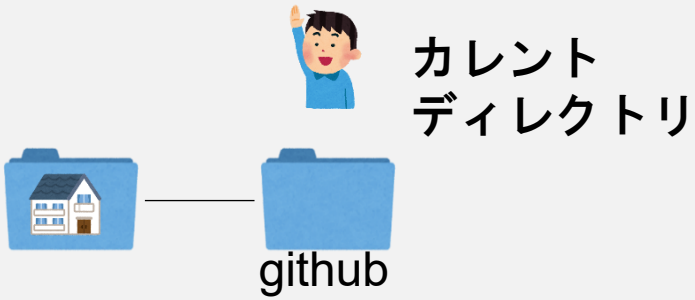
ホームディレクトリ

.gitconfig

課題 – Step 2

ディレクトリの作成

```
cd
mkdir github
cd github
```



ホームディレクトリに移動してから
gitディレクトリを作成し、そこへ移動

```
mkdir test
cd test
```

さらにtestディレクトリを作成して
そこへ移動

課題 – Step 2

Git Bashを開いた直後は「ホームディレクトリ」がカレントディレクトリになっている (PC室の環境では/z/になっているはず)

```
watanabe@DESKTOP-ASHEQTP MINGW64 ~
$ mkdir github

watanabe@DESKTOP-ASHEQTP MINGW64 ~
$ cd github

watanabe@DESKTOP-ASHEQTP MINGW64 ~/github
$
```

ディレクトリを作成

「cd github」コマンドにより、カレントディレクトリをgithubに移動すると右の表示が変わる(/z/githubになるはず)

Git Bashのプロンプト

Git Bashはカレントディレクトリとカレントブランチを表示する

Git Bashを開いた直後は「ホームディレクトリ」がカレントディレクトリになっている

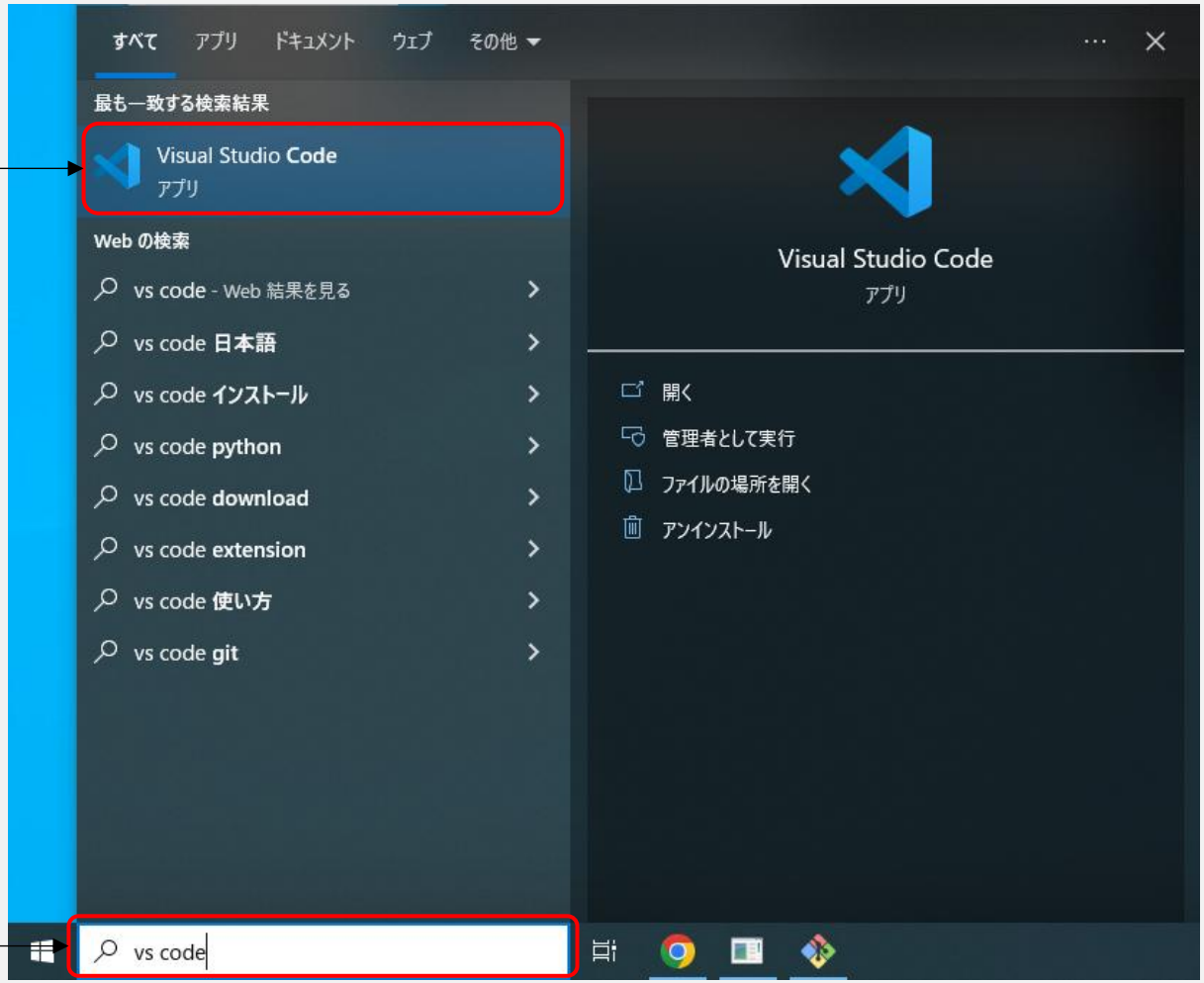
```
watanabe@DESKTOP-ASHEQTP MINGW64 ~
$ cd github

watanabe@DESKTOP-ASHEQTP MINGW64 ~/github
$
```

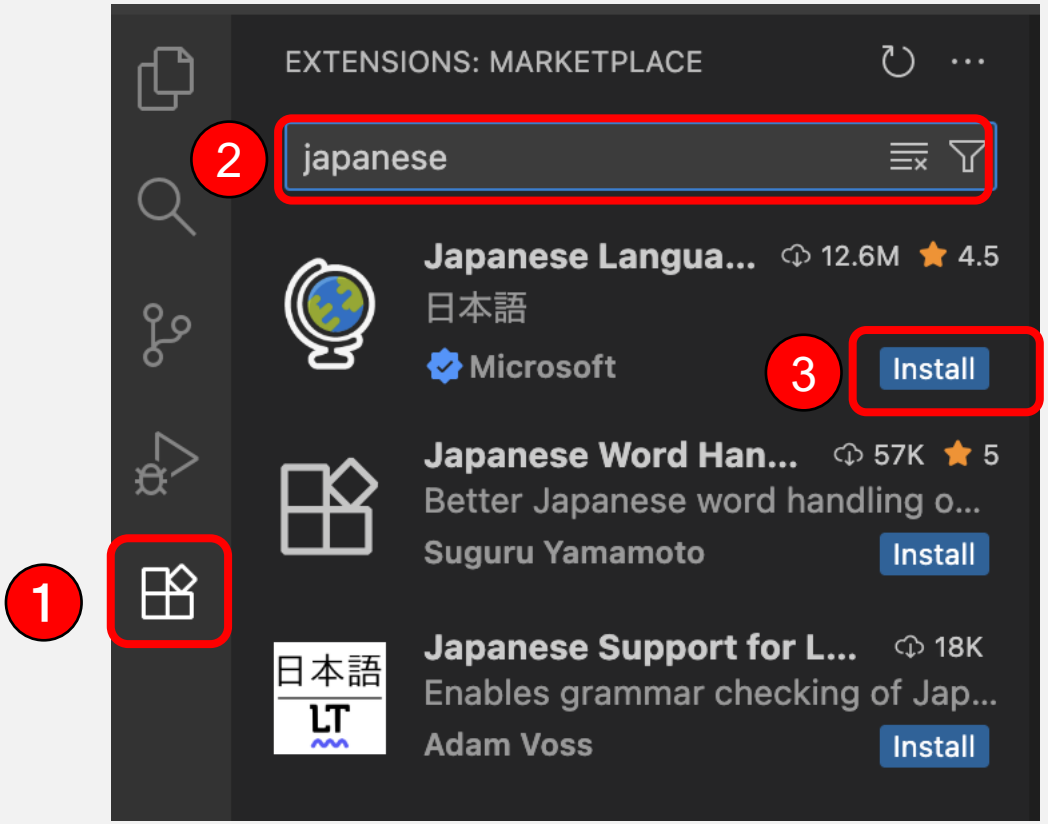
「cd github」コマンドにより、カレントディレクトリをgithubに移動すると右の表示が変わる

VS Codeの起動

1. 検索ボックスにvs codeと入力
2. 表示された「Visual Studio Code」をクリック



VS Codeの日本語化

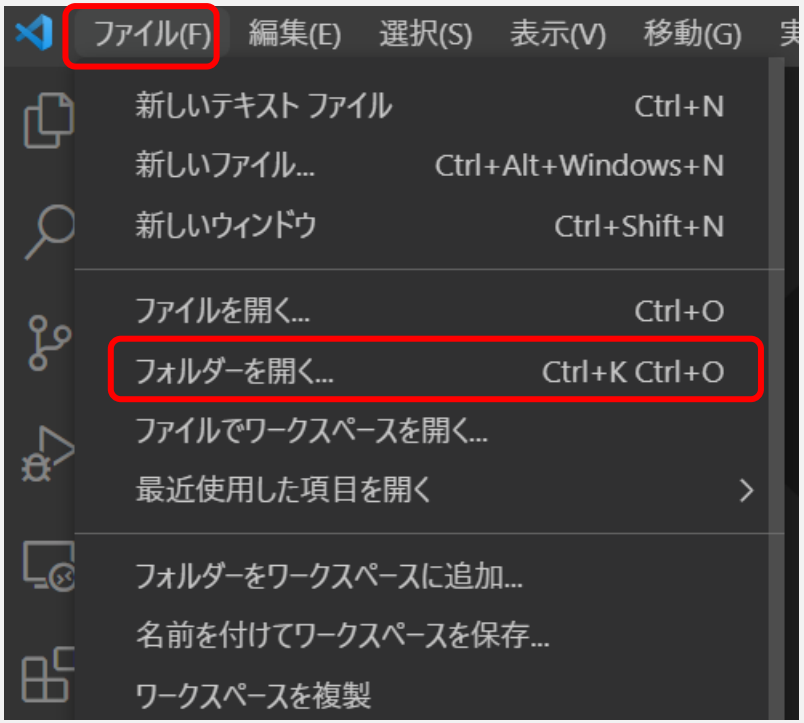


1. 左の「拡張機能」アイコンをクリック
2. 現れた検索窓に「japanese」と入力
3. 「Japanese Language Pack for VS Code」をインストールして再起動

VS Code

フォルダの開き方

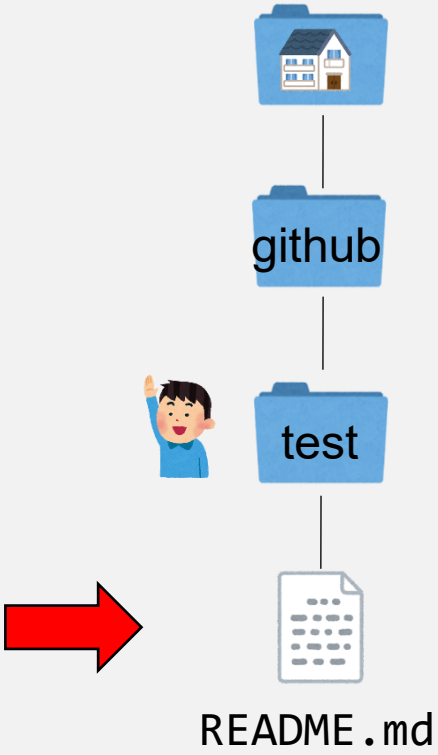
「ファイル」から「フォルダーを開く」を選択



「このフォルダー内のファイルの作成者を信頼しますか？」という表示が出たら「はい、作成者を信頼します」を選ぶ

課題 – Step 2

README.mdファイルの作成



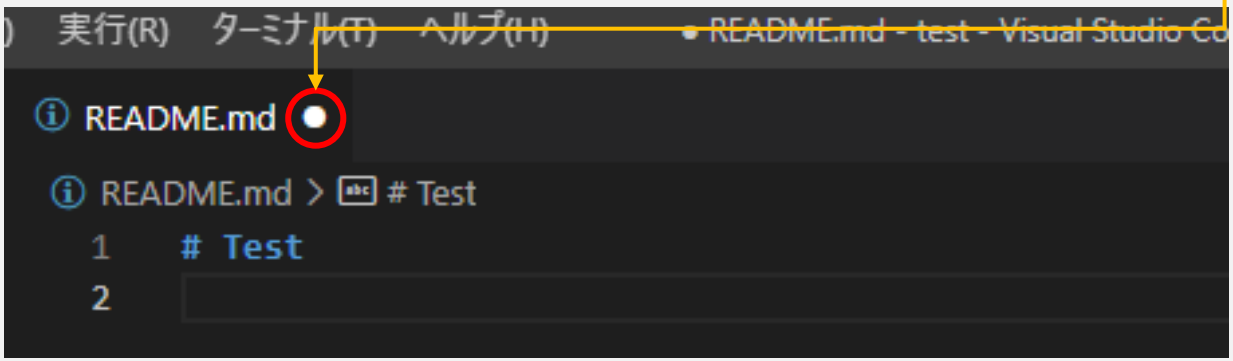
課題 – Step 2

1. ファイルの新規作成ボタンを押して
2. 「README.md」と入力

課題 – Step 2

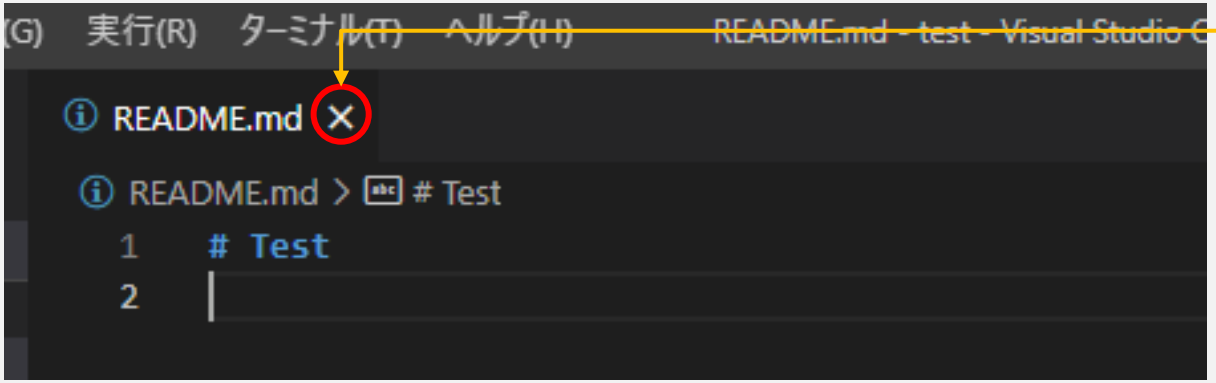
ファイル内容を入力したら保存すること

ファイル名のタブの隣のアイコンが「●」だと未保存



Ctrl+S
もしくは
「ファイル」の
「保存」をクリック

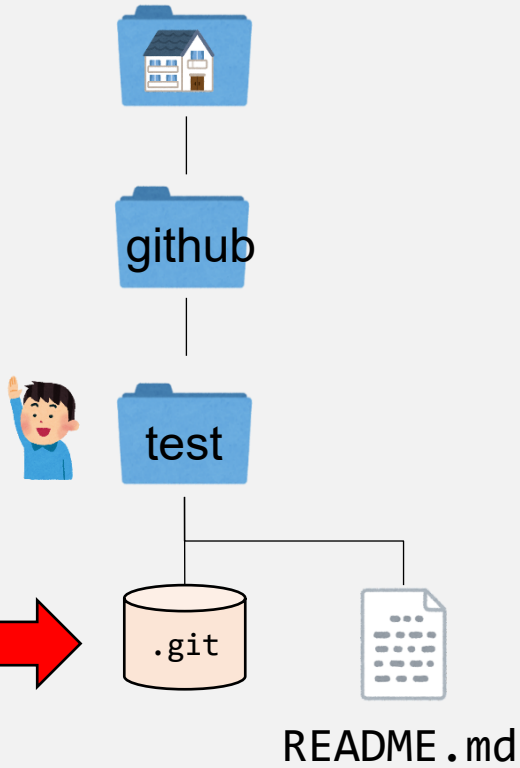
ファイル名のタブの隣のアイコンが「X」だと保存済み



課題 – Step 2

リポジトリの初期化

```
git init
```



.gitディレクトリが作られ、
testがgit管理下に入る

Git Bashとカレントブランチ

Git Bashは、カレントディレクトリがGitリポジトリでない場合は
カレントディレクトリしか表示しない

```
watanabe@DESKTOP-ASHEQTP MINGW64 ~/github/test
$ git init
Initialized empty Git repository in C:/Users/watanabe/github/test/.git/

watanabe@DESKTOP-ASHEQTP MINGW64 ~/github/test (main)
$
```

カレントディレクトリがGitリポジトリの場合は、カレントブランチを表示する

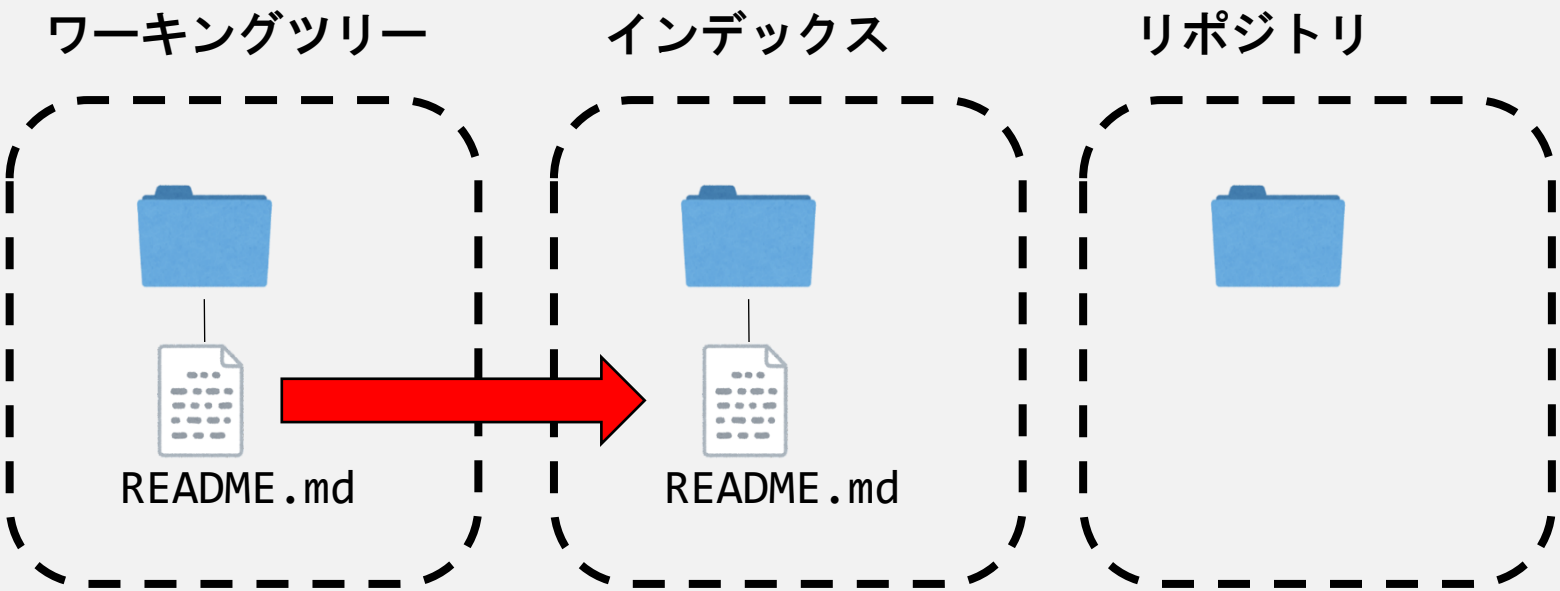
プロンプトを見て、カレントディレクトリとカレントブランチを常に確認すること

課題 – Step 3

インデックスへの追加

```
git add README.md
```

ワーキングツリーからインデックスにファイルをコピー

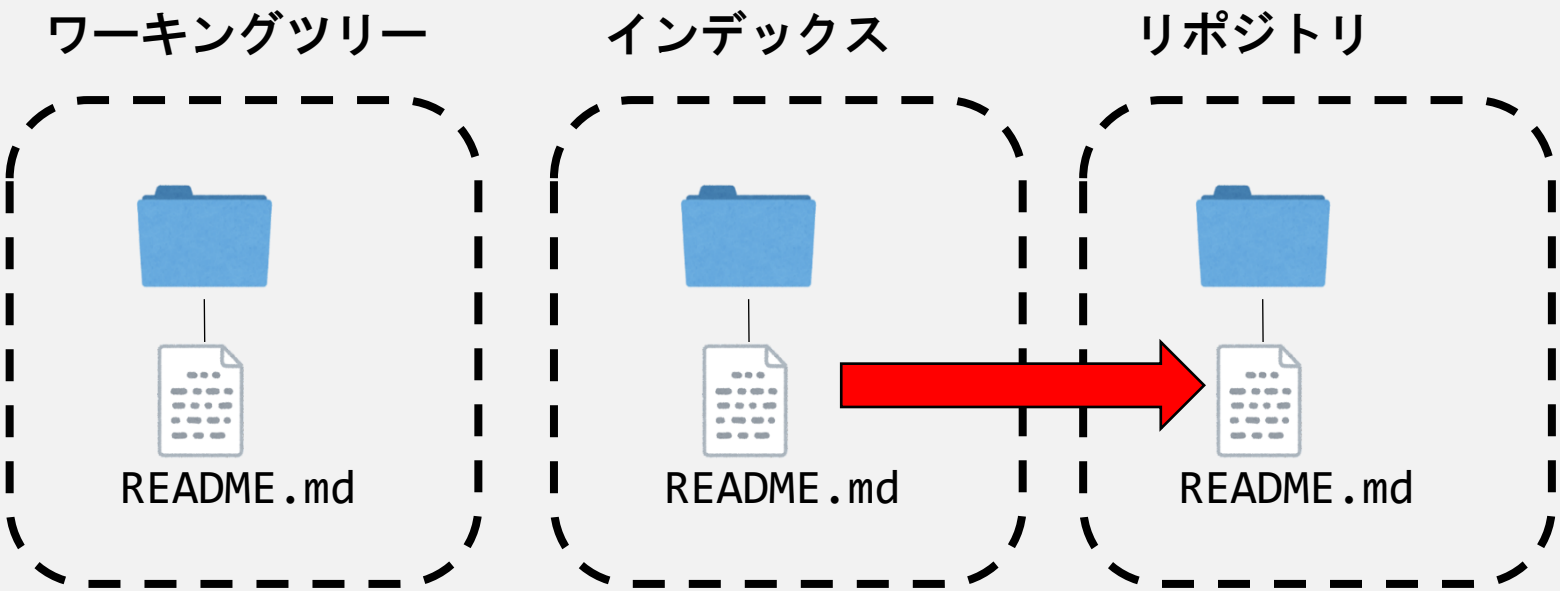


課題 – Step 3

コミット

```
git commit -m "initial commit"
```

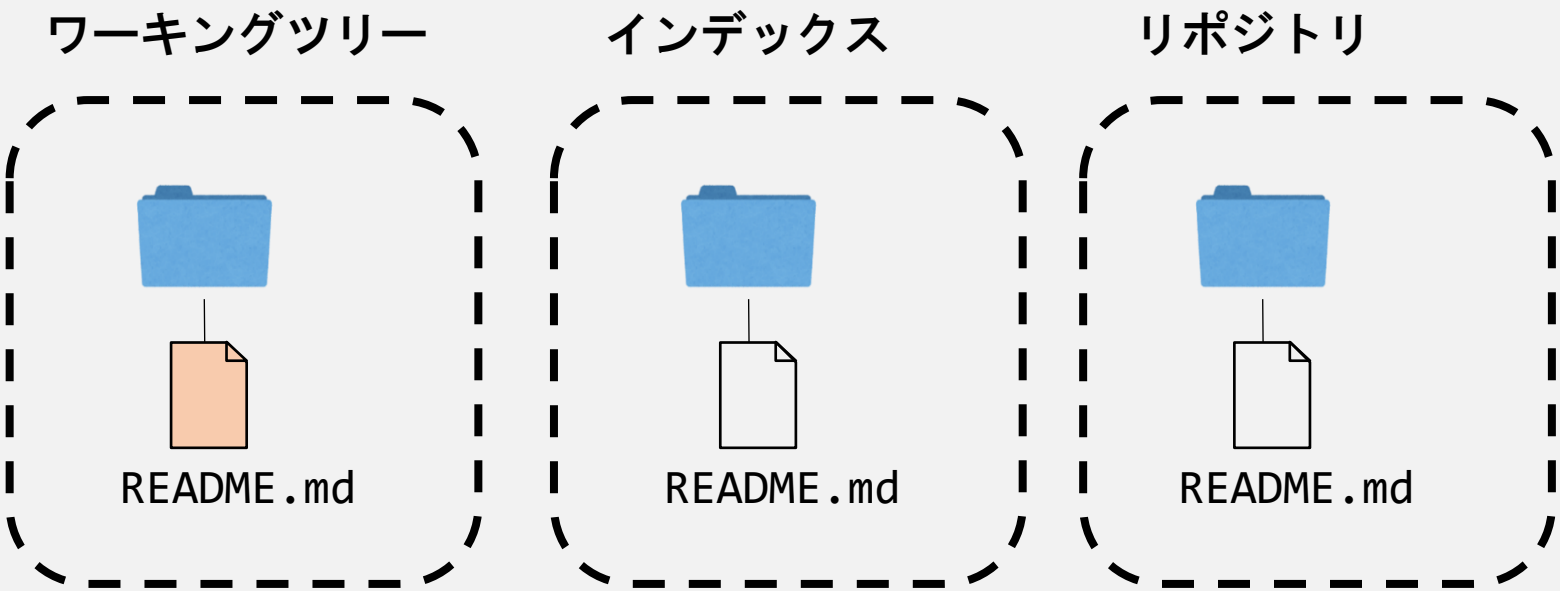
インデックスの状態をそのまま「歴史」としてリポジトリに保存



課題 – Step 4

ファイルの修正

ワーキングツリーが、インデックス、リポジトリと異なる状態になる

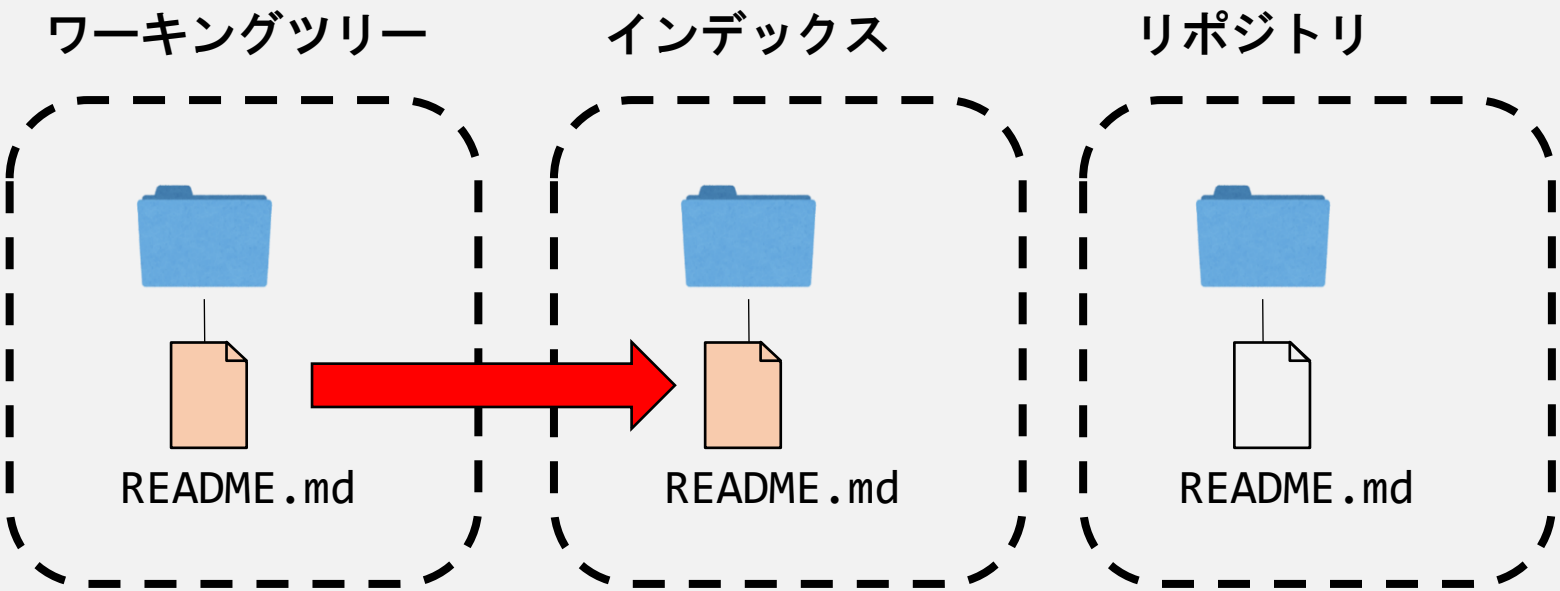


課題 – Step 4

インデックスへの追加

```
git add README.md
```

ワーキングツリーからインデックスにファイルをコピー

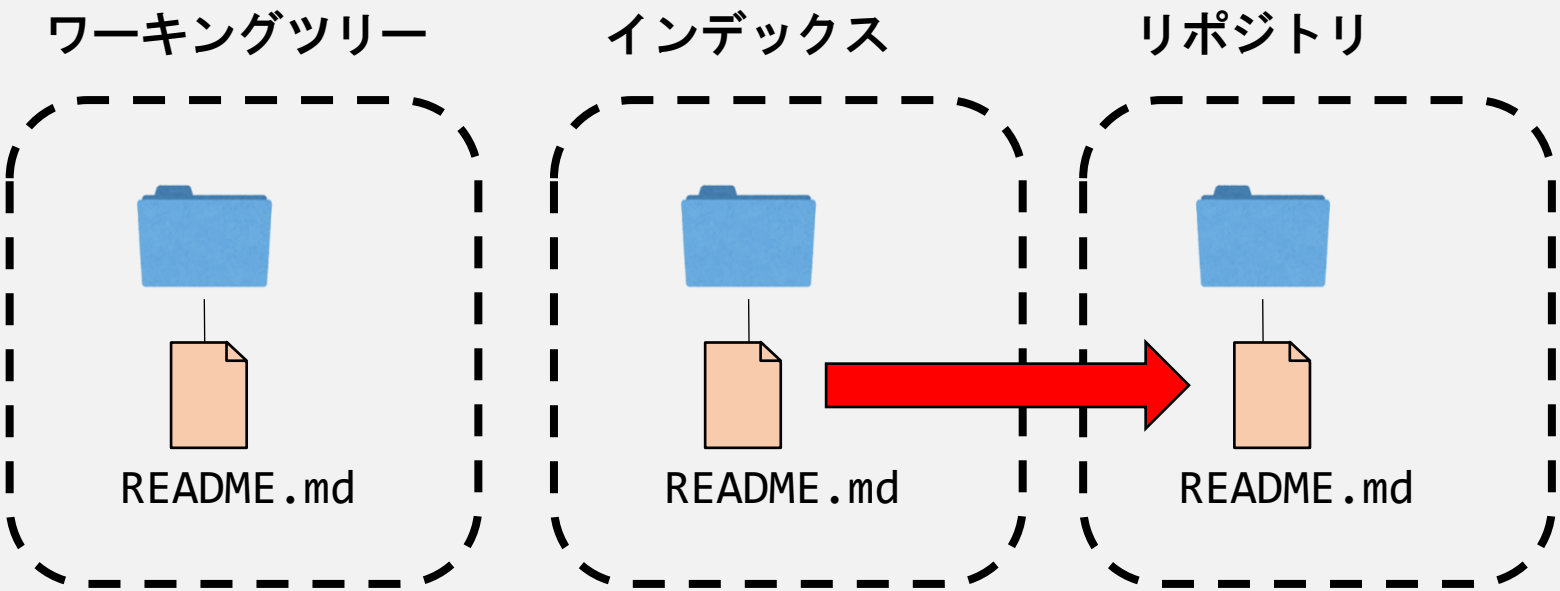


課題 – Step 4

コミット

```
git commit -m "adds new line"
```

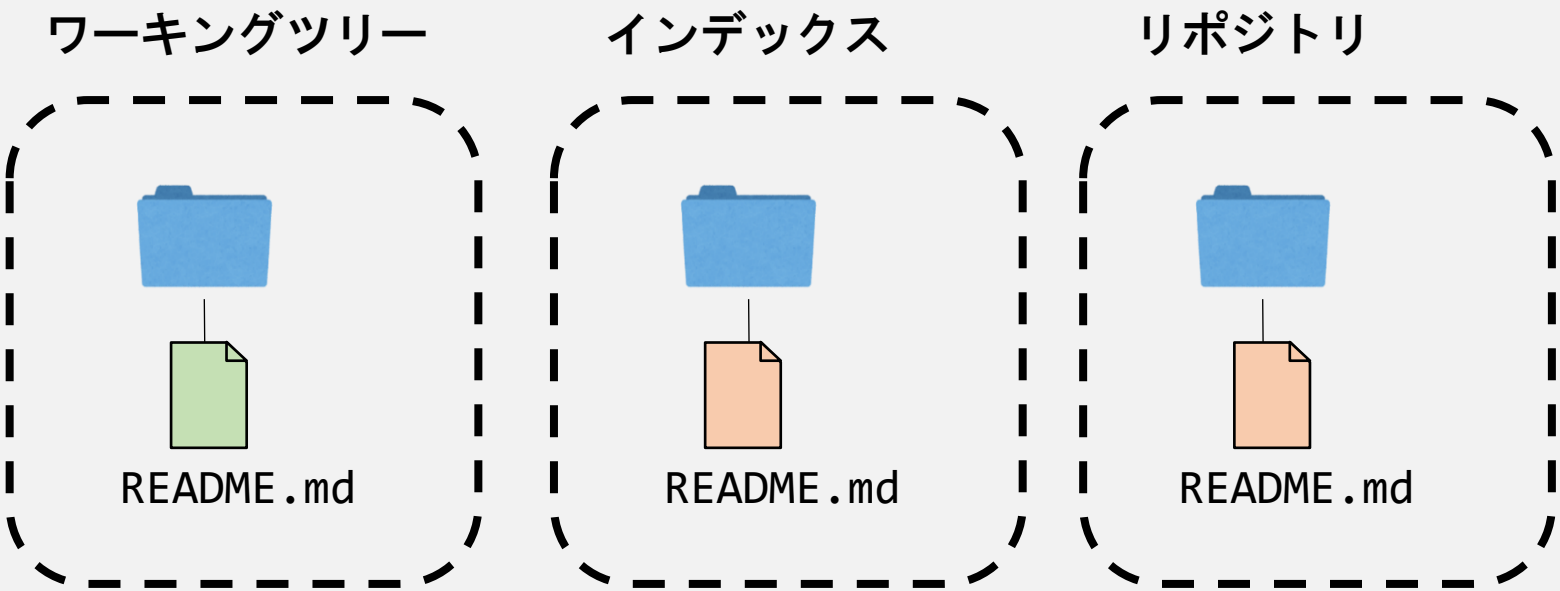
インデックスの状態をそのまま「歴史」としてリポジトリに保存



課題 – Step 5

ファイルの修正

ワーキングツリーが、インデックス、リポジトリと異なる状態になる

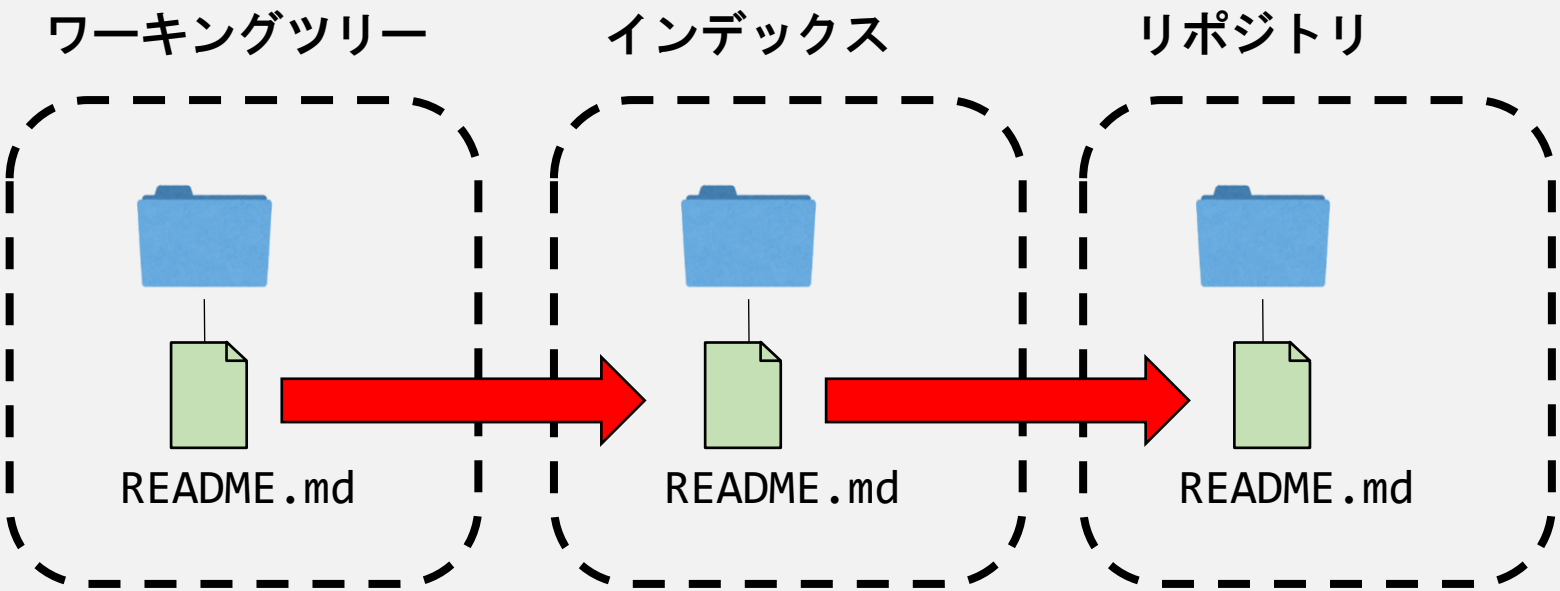


課題 – Step 5

コミット(自動ステージング)

```
git commit -am "modifies README.md"
```

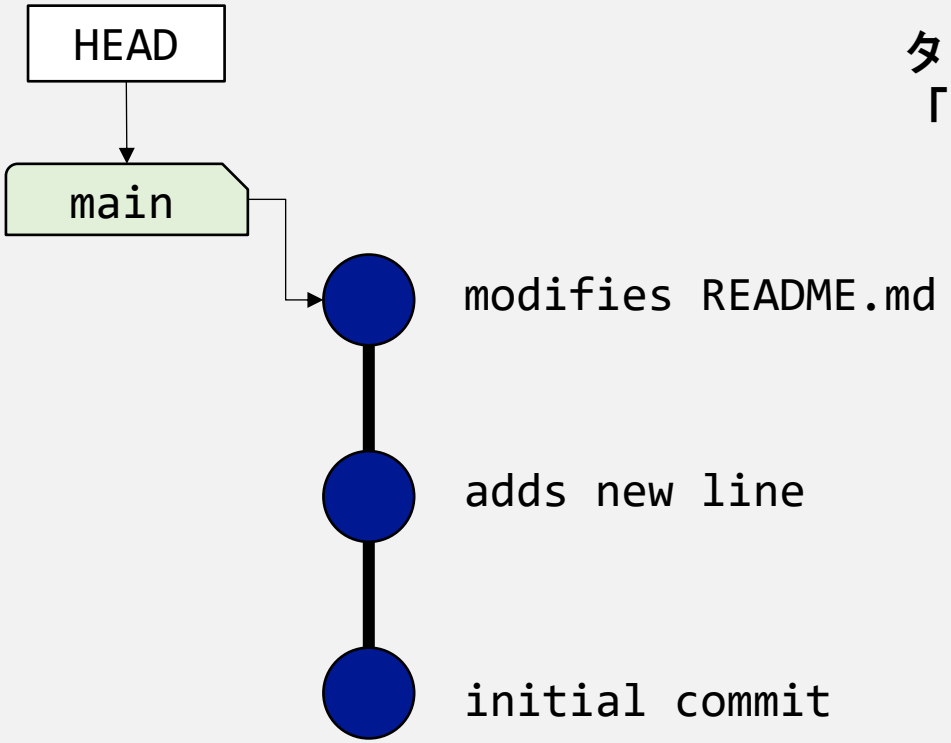
commit -a オプションにより、修正のあるファイル全てを
自動でステージングしてからコミットする
(ステージングを飛ばしているわけではない)



課題 – Step 6

歴史の確認

```
git log
git log --oneline
```



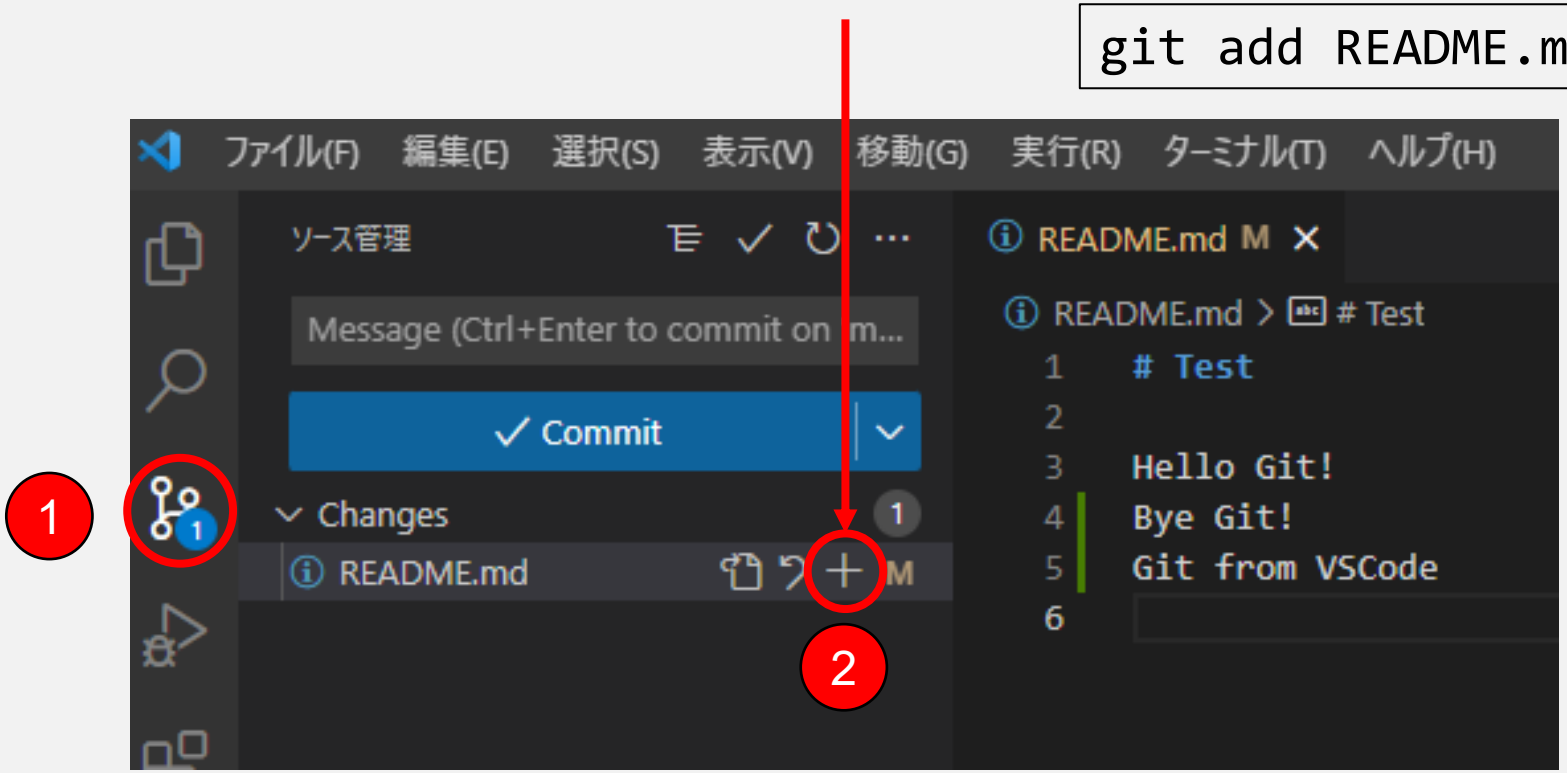
ターミナルに表示された情報から
「玉と線」を頭の中に再構成すること

課題 – Step 7

VSCodeからのコミット

この「＋」ボタンがgit addに対応

```
git add README.md
```

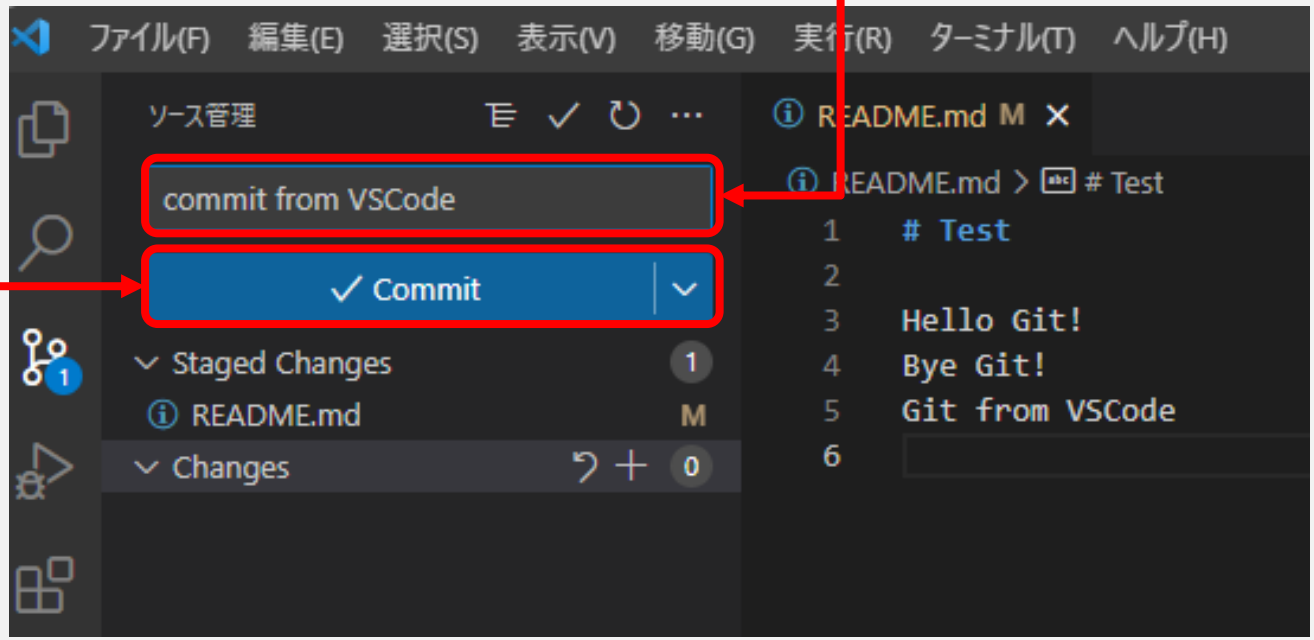


課題 – Step 7

```
git commit -m “commit from VSCode”
```

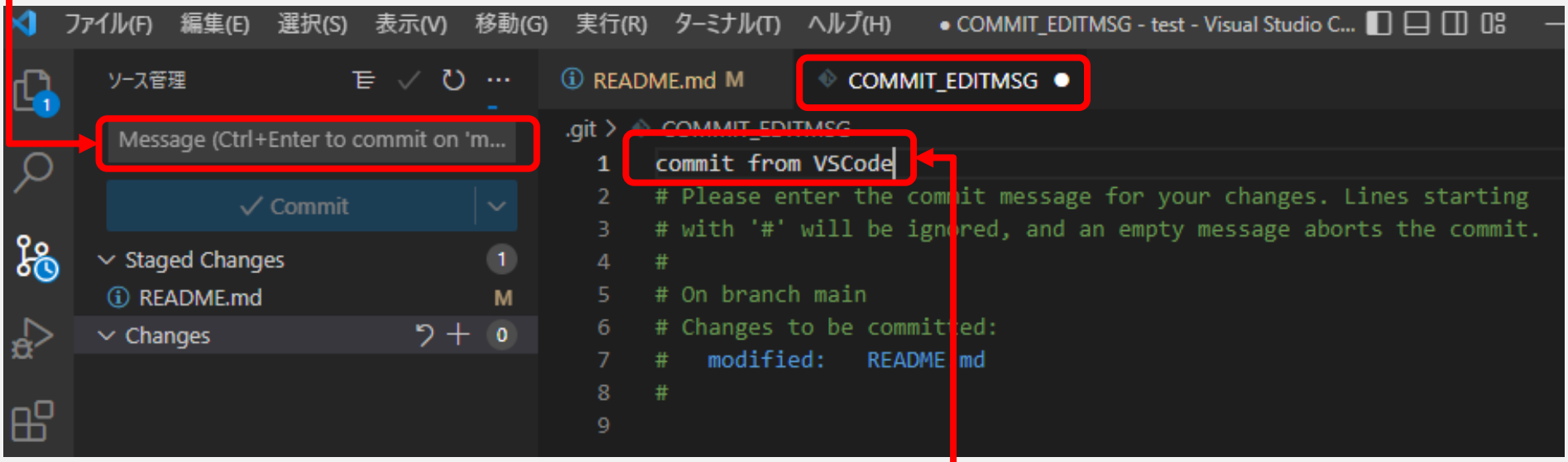
コミットボタン

ここにコミットメッセージが書ける



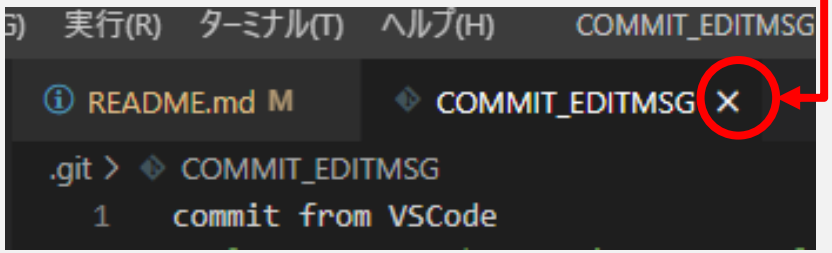
課題 – Step 7

もしコミットメッセージを書き忘れてコミットボタンを押したら
COMMIT_EDITMSGというファイルが開く

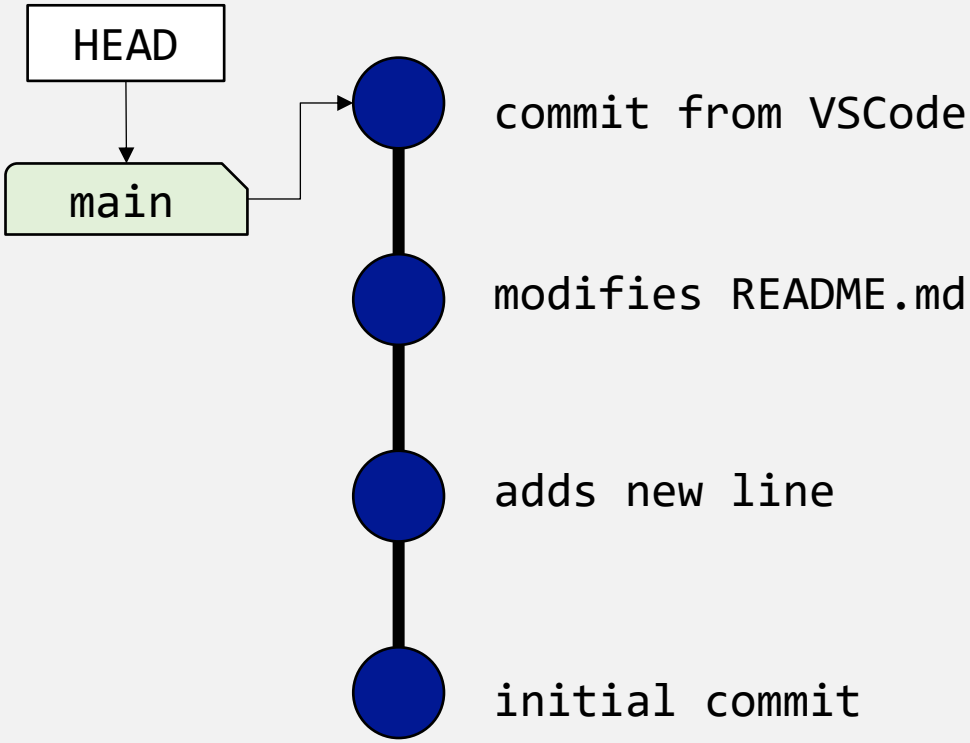


一番上に「commit from VSCode」と入力する

保存してからファイルを閉じる



課題 – 最終確認

歴史の確認

```
git log --oneline
```